

Tarea # 2

Grupo #10

COMUNICACIÓN ENTRE MICRO CONTROLADORES

ATMEGA328P-PIC16F887

Profesor:

Ing. Ronald Solís

Estudiante:

Bazán Jostin

Durango Ana Belén

García Clear

Lozano Enna

Pogo Diego

Paralelo: 1

Fecha de Entrega:

09 de Junio del 2026

I – PAO – 2026

1. Objetivos

1.1 Objetivo General

Desarrollar un sistema de juego interactivo de 3 niveles de dificultad que combine salidas visuales mediante una matriz LED 8x8 controlada por el microcontrolador ATmega328P, y salidas auditivas mediante un sistema de reproducción de melodías con el microcontrolador PIC16F887, y entradas mediante pulsadores, empleando comunicación entre ambos dispositivos y simulando el funcionamiento completo en Proteus, con el propósito de reforzar habilidades en programación y control de sistemas embebidos. El sistema deberá ser programado en C para ambos microcontroladores.

1.2 Objetivos Específicos

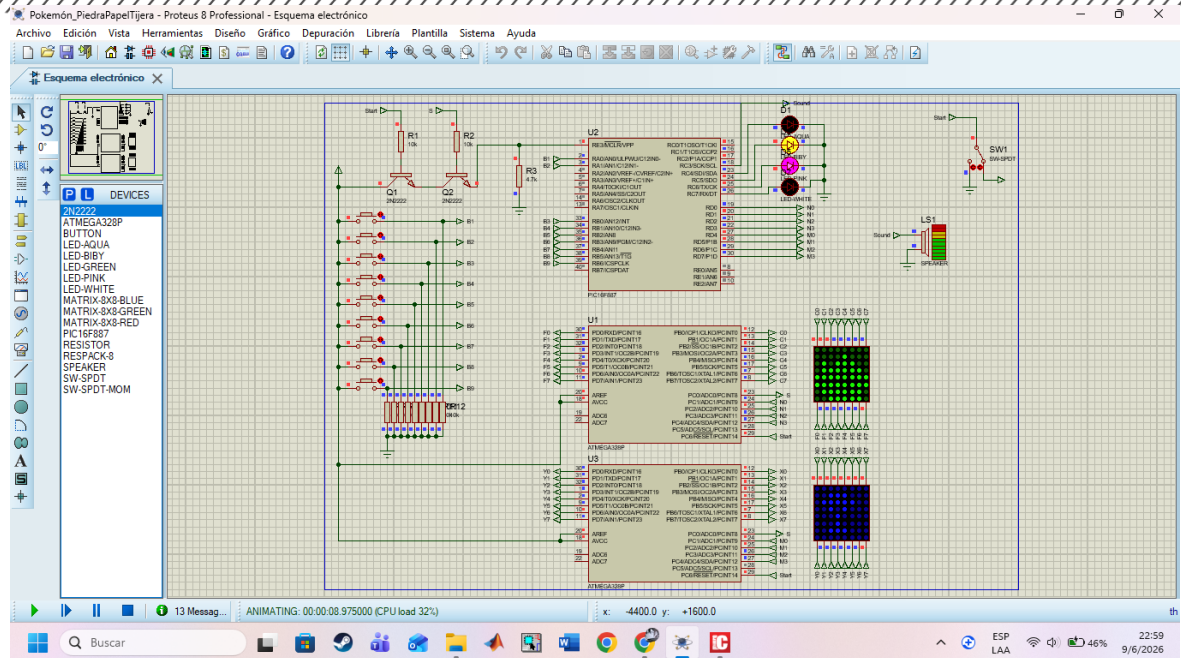
El programa desarrollado en lenguaje C para un microcontrolador AVR tiene como finalidad controlar una matriz de LEDs de 8x8 para mostrar diferentes figuras según las señales de entrada recibidas. Para ello, se configuran los puertos del microcontrolador, donde PORTB y PORTD se utilizan como salidas para manejar las columnas y filas de la matriz, mientras que PORTC se emplea como entrada para leer datos externos y señales de control. El sistema trabaja con una frecuencia de 8 MHz, lo que permite realizar un barrido rápido de la matriz y generar una visualización estable mediante multiplexación.

2. Descripción del juego

El funcionamiento del programa se basa en la lectura de un valor binario proveniente de los pines PC1 a PC4, el cual es convertido a un número decimal. Dependiendo de este valor y del estado del pin PC5, el sistema determina qué imagen debe mostrarse en la matriz. Si el valor leído corresponde a 0x0F, el sistema entra en un modo de espera y muestra un símbolo de interrogación. En cambio, si el pin PC5 se encuentra activado, el programa interpreta que se ha obtenido un resultado positivo y despliega uno de los nueve gráficos definidos, que representan distintos elementos como agua, fuego, planta, eléctrico, tierra, roca, hielo, acero y dragón. Por otro lado, si el pin PC5 no está activo y el valor leído es distinto de cero, el sistema muestra una "X", indicando un estado de derrota o empate.

La visualización de las imágenes se realiza mediante una función que recorre rápidamente cada fila de la matriz LED, activando una a la vez mientras envía la información correspondiente a las columnas. Este proceso se repite múltiples veces en un corto intervalo de tiempo, logrando así que el ojo humano perciba una imagen completa y estable. Cada figura está definida como un arreglo de ocho bytes, donde cada byte representa una fila de la matriz.

3. Simulación en Proteus



4. Código y Explicación ATmega328P:

```

5  #define START_PIC PC0
6
7  unsigned char signo[8] = {0x00, 0x04, 0x02, 0x01, 0xB1, 0x0A, 0x04, 0x00}; //?
8  unsigned char PORT[8] = {1, 2, 4, 8, 16, 32, 64, 128};
9
10 // Elementos
11 unsigned char AGUA[8] = {0x30, 0x78, 0xFC, 0xFE, 0xFC, 0x78, 0x30, 0x00};
12 unsigned char FUEGO[8] = {0x30, 0x7C, 0xF8, 0xFC, 0xFE, 0xFC, 0xF8, 0x30};
13 unsigned char PLANTA[8] = {0x00, 0x04, 0x4E, 0x76, 0x4E, 0x04, 0x00, 0x00};
14 unsigned char ELECTRICO[8] = {0x00, 0x08, 0x0C, 0x6E, 0x3B, 0x18, 0x08, 0x00};
15 unsigned char TIERRA[8] = {0x70, 0xFC, 0x76, 0x07, 0x76, 0xFC, 0x70, 0x00};
16 unsigned char ROCA[8] = {0x00, 0x7E, 0x7E, 0x7E, 0x7E, 0x7E, 0x7C, 0x00};
17 unsigned char HIELO[8] = {0x60, 0x60, 0x18, 0x78, 0x60, 0x18, 0x78, 0x60};
18 unsigned char ACERO[8] = {0x38, 0x6C, 0x46, 0x02, 0x02, 0x46, 0x6C, 0x38};
19 unsigned char DRAGON[8] = {0x03, 0x46, 0x64, 0x18, 0x18, 0x64, 0x46, 0x03};

```

Al inicio se definen una constante que usaremos `START_PIC`, además de los varios símbolos que se usarán en el juego, un “?” para el inicio del juego antes de la elección, una “X” para el momento que el jugador pierde y los 9 símbolos con los que se jugarán.

```

24 void mostrarMatriz(int side, const unsigned char* imagen) {
25     if (side == 8) {
26         for (int k = 0; k < 50; k++) {
27             for (int j = 0; j < 8; j++) {
28                 PORTD = PORT[j];
29                 PORTB = ~imagen[j];
30                 _delay_ms(0.05);
31             }
32         }
33     }
34 }

```

La función `mostrarMatriz()` sirve para mostrar los símbolos en una matriz de 8 puntos a lo largo del juego.

```
36 unsigned char binarioADecimal() {
37     return ((PINC & 0x1E) >> 1);
38 }
```

Esta función es para el ingreso del número que se encuentra en los pines PC1 a PC5, primero con el AND se ponen en 0 todos los pines que no usamos y desplazamos el número un bit a la derecha para ignorar el PC0 que no es parte del número. Este número en binario representa el botón físico que el usuario presiona para elegir el tipo que escogió.

```
38 int main(void) {
39     DDRB = 0xFF; DDRD = 0xFF; DDRC = 0x01; PORTC = 0x00;
40     PORTC |= (1 << START_PIC);
41
42     while (1) {
43         unsigned char numero = binarioADecimal();
44
45         if (numero == 0x0F || numero == 0) {
46             // Si lee 15 (0x0F) se queda en espera mostrando el "?"
47             mostrarMatriz(8, signo);
48         }
49         else {
50             // Muestra el elemento
51             switch (numero) {
52                 case 1: mostrarMatriz(8, AGUA); break;
53                 case 2: mostrarMatriz(8, FUEGO); break;
54                 case 3: mostrarMatriz(8, PLANTA); break;
55                 case 4: mostrarMatriz(8, ELECTRICO); break;
56                 case 5: mostrarMatriz(8, TIERRA); break;
57                 case 6: mostrarMatriz(8, ROCA); break;
58                 case 7: mostrarMatriz(8, HIELO); break;
59                 case 8: mostrarMatriz(8, ACERO); break;
60                 case 9: mostrarMatriz(8, DRAGON); break;
61                 default: mostrarMatriz(8, signo); break;
62             }
63         }
64     }
65     return 0;
66 }
```

El Programa principal del ATmega, primero se definen los pines de entrada y salidas. En el ciclo `while()` el programa pregunta por el número escogido por el PIC, si el número es 15 o 0 el programa se mantiene en espera. Si no, al presionar un botón sea que el usuario gane o pierda se mostrará en la matriz el tipo elegido con el botón presionado. Esto funciona igual para el ATmega del usuario y de la máquina.

PIC16F887:

```
unsigned short numSelec = 0;
unsigned short randomNumber = 0;
unsigned short nivel = 1;
unsigned short victoriasSeguidas = 0;
```

Variables en el PIC que representan el tipo elegido por el usuario, el tipo elegido por la máquina, el nivel en que se juega y la cantidad de victorias seguidas, que es condición para subir de nivel, respectivamente.

```
// Melodías
void nota_C5(int duration) { Sound_Play(523.25, duration); }
void nota_E5(int duration) { Sound_Play(659.25, duration); }
void nota_G5(int duration) { Sound_Play(783.99, duration); }
void nota_C6(int duration) { Sound_Play(1046.50, duration); }

void nota_D5_sharp(int duration) { Sound_Play(622.25, duration); }
void nota_D5(int duration) { Sound_Play(587.33, duration); }
void nota_C5_sharp(int duration) { Sound_Play(554.37, duration); }
void nota_B4(int duration) { Sound_Play(493.88, duration); }
```

Las notas que se usarán en las melodías de victoria, derrota y subir nivel usando la función Sound_Play().

```
void Melody_Win() {
    nota_C5(200); Delay_ms(50);
    nota_E5(200); Delay_ms(50);
    nota_G5(200); Delay_ms(50);
    nota_C6(350);
}

void Melody_Lose() {
    nota_D5_sharp(250); Delay_ms(50);
    nota_D5(250); Delay_ms(50);
    nota_C5_sharp(250); Delay_ms(50);
    nota_B4(400);
}

void Sound_LevelUp() {
    Sound_Play(523.25, 150); Delay_ms(50);
    Sound_Play(659.25, 150); Delay_ms(50);
    Sound_Play(783.99, 300);
}
```

Las 3 funciones que componen las 3 melodías que utilizamos con el PIC.

```
// Matriz de resultados (0=Empate, 1=Victoria, 2=Derrota)
const unsigned short REGLAS[10][10] = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    {0, 0, 1, 2, 2, 1, 1, 1, 1, 2}, //Agua
    {0, 2, 0, 1, 1, 2, 2, 1, 1, 2}, //Fuego
    {0, 1, 2, 0, 1, 1, 2, 1, 2, 2}, //Planta
    {0, 1, 2, 2, 0, 2, 2, 1, 1, 2}, //Eléctrico
    {0, 2, 1, 2, 1, 0, 1, 1, 1, 2}, //Tierra
    {0, 2, 1, 1, 1, 2, 0, 1, 2, 1}, //Roca
    {0, 2, 2, 1, 2, 2, 2, 0, 2, 1}, //Hielo
    {0, 2, 2, 1, 2, 2, 1, 1, 0, 1}, //Acero
    {0, 1, 1, 1, 1, 1, 2, 2, 2, 0} //Dragón
};
```

La matriz de resultados con los empates (0), victorias (1) y derrotas (2) según el tipo escogido.

```
void main() {
    ANSEL = 0; ANSELH = 0;
    C1ON_bit = 0; C2ON_bit = 0;

    TRISB = 0xFF; TRISA = 0xFF;
    TRISC = 0x00; TRISD = 0x00;

    Sound_Init(&PORTC, 3);
    srand(1);

    while (1) {
        numSelec = 0;

        //ESTADO DE ESPERA
        PORTD = 0xFF; // Ambos reciben 0x0F para pintar el "?"
    }
}
```

Inicio del programa principal del PIC, como utilizamos 2 ATmega el PIC envía 0xFF en sus 8 bits y a cada ATmega envía 0x0F para mantenerlo en estado de espera.

```
//ESCANEAO DE BOTONES FILTRADO POR NIVEL
if (Button(&PORTA, 0, 1, 1)) numSelec = 1; while(RA0_bit);
if (Button(&PORTA, 1, 1, 1)) numSelec = 2; while(RA1_bit);
if (Button(&PORTB, 0, 1, 1)) numSelec = 3; while(RB0_bit);

if (nivel >= 2) {
    if (Button(&PORTB, 1, 1, 1)) numSelec = 4; while(RB1_bit);
    if (Button(&PORTB, 2, 1, 1)) numSelec = 5; while(RB2_bit);
    if (Button(&PORTB, 3, 1, 1)) numSelec = 6; while(RB3_bit);
}

if (nivel == 3) {
    if (Button(&PORTB, 4, 1, 1)) numSelec = 7; while(RB4_bit);
    if (Button(&PORTB, 5, 1, 1)) numSelec = 8; while(RB5_bit);
    if (Button(&PORTB, 6, 1, 1)) numSelec = 9; while(RB6_bit);
}
```

Filtrado de botones, dependiendo del nivel solo se permite el ingreso de ciertos botones. Al nivel 1 solo se habilitan RA0, RA1 y RB0, al nivel 2 se habilita hasta RB3 y al nivel 3 hasta RB6, esto para que solo aumenten los tipos al nivel 3.

```
//ROCESAR COMBATE
if (numSelec != 0) {
    if (nivel == 1)        randomNumber = 1 + rand() % 3;
    else if (nivel == 2)  randomNumber = 1 + rand() % 6;
    else                  randomNumber = 1 + rand() % 9;

    PORTD = (numSelec & 0x0F) | ((randomNumber & 0x0F) << 4);

    if (REGLAS[numSelec][randomNumber] == 1) {
        //VICTORIA: Sube racha y evalúa nivel
        victoriasSeguidas++;
        if (victoriasSeguidas >= 3 && nivel < 3) {
            nivel++;
            victoriasSeguidas = 0;
            Sound_LevelUp();
        } else {
            Melody_Win();
        }
        Delay_ms(2500);
    }
    else if (REGLAS[numSelec][randomNumber] == 2) {
        //DERROTA: Se destruye la racha
        victoriasSeguidas = 0;
        Melody_Lose();
        Delay_ms(2500);
    }
    else {
        //EMPATE: La racha permanece intacta
        Delay_ms(2500);
    }
}
```

Finalmente, la lógica del juego, dependiendo del nivel y a través de la función rand() con un operador de módulo (%) la máquina elegirá solo tipos de acuerdo con el nivel actual. Una vez se presiona un botón y se eligió un tipo buscamos su índice y el índice del tipo elegido por la máquina en la matriz de reglas, si el resultado es 1, victoria, se sumará 1 a la variable victoriasSeguidas y si está es igual a 3 se aumentará un nivel. Si el valor en la matriz reglas es 2 se resetea el contador de victorias seguidas y cuenta como derrota. Si el valor es 0 solo se muestra los tipos en las matrices de puntos sin cambios en las victorias seguidas.

5. Github: <https://github.com/nlozanor/TAREA2-EMBEBIDOS.git>

6. Conclusiones y Recomendaciones

En conclusión, este programa demuestra el uso de técnicas de multiplexación para el control de matrices LED, así como la interacción entre entradas digitales y salidas visuales en un sistema embebido, permitiendo representar distintos estados de un sistema de manera gráfica y eficiente.